AUTHENTICITY VALIDATOR FOR ACADEMIA

Submitted for the partial fulfilment of the requirements
For the award of the degree in

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

SUBMITTED BY

LOURDU TYTUS GLASTIN R

(DEPT NO:23-UCS-026)

Under The Guidance Of

Prof. ANTONY S. ALEXANDER

ASSISTANT PROFESSOR

DEPARTMENT OF COMPUTER SCIENCE AND APPLICATIONS

LOYOLA COLLEGE (AUTONOMOUS)

CHENNAI – 600 034

MARCH – 2026

DECLARATION

This is to declare that the project report entitled "**Authenticity Validator For Academia**" is carried out by **Lourdu Tytus Glastin R (23-UCS-026)** under the guidance of **Prof. Antony S. Alexander** at Loyola College (Autonomous), Chennai – 34, and is being submitted in partial fulfilment of the requirements for the award of the degree **BACHELOR OF SCIENCE IN COMPUTER SCIENCE.**

Date: Signature of the Candidate:

BONAFIDE CERTIFICATE

This is to certify that the project work titled "**Authenticity Validator For Academia**" submitted to Loyola College, Chennai in partial fulfilment of the requirements for the award of the degree BACHELOR OF SCIENCE IN COMPUTER SCIENCE is a record of original project work carried out by **Lourdu Tytus Glastin R (23-UCS-026)** during the period 2025 – 2026 of study in the Department of Computer Science, Loyola College, Chennai, under the supervision and guidance of **Prof. Antony S. Alexander, M.C.A., M.Phil., M.Sc.,** Department of Computer Science, Loyola College, Chennai.

Head of the Department	Project Guide
Dr. J. JERALD INICO	PROF. ANTONY S. ALEXANDER
M.Sc., M.Phil., MCP., Ph.D.	M.C.A., M.Phil., M.Sc.,

Submitted for the Viva-Voce Examination held on 28-03-2026

Examiner 1 Examiner 2

AUTHENTICITY VALIDATOR FOR ACADEMIA

ABSTRACT

The Authenticity Validator for Academia is a web-based application developed to verify the originality and integrity of academic documents such as assignments, research papers, and theses. With the increasing use of digital tools and artificial intelligence in content creation, maintaining academic honesty has become more challenging. This system addresses the issue by combining Natural Language Processing (NLP) and Machine Learning (ML) techniques to perform both plagiarism detection and AI-generated content identification within a single platform.

The application analyzes uploaded documents by extracting and preprocessing text, then applying TF-IDF vectorization and cosine similarity to identify similarities with existing documents. This enables the detection of both direct copying and paraphrased content. In addition, the system evaluates linguistic features such as sentence structure, repetition, and lexical diversity to estimate the probability of the text being AI-generated. By integrating these two methods, the system provides a more comprehensive approach compared to traditional plagiarism checkers.

The platform is built using a full-stack architecture, with a frontend developed using HTML, Bootstrap, and Chart.js for an interactive user experience, and a backend powered by Python and Flask for processing and analysis. SQLAlchemy is used for database management, while tools like Flask-JWT-Extended and Werkzeug ensure secure authentication and data handling. The system generates detailed reports, including similarity scores, AI probability metrics, and visual representations such as charts, making it easier for users and administrators to interpret the results.

Overall, the Authenticity Validator provides an efficient, reliable, and user-friendly solution for academic content verification. It reduces manual effort, improves accuracy, and promotes academic integrity in an environment increasingly influenced by AI-driven content creation.

ACKNOWLEDGEMENT

I extend my deepest gratitude and appreciation to all those who have contributed to the successful completion of my project. This endeavour has been a valuable learning experience, and I am indebted to several individuals and organizations for their support.

I would like to extend my heartfelt gratitude to **Rev. Dr. A. LOUIS AROKIARAJ SJ**, the Principal, Loyola College and my special gratitude I bestowed **Mr. L. JOSEPH AROCKIASAMY,Y** the Deputy Principal, Loyola College for his constant encouragement.

I would like to thank **DR. J. JERALD INICO M.SC., M. C. P., M.Phil., Ph.D.,** Head of the Department, Department of Computer Science and Applications, Loyola College and **Dr. I. JUSTINSOPHIA M.Sc., M.Phil., Ph.D.,** Coordinator, Department of Computer Science and Applications, Loyola College, for their support in carrying out my project.

I sincerely express my gratitude to our project guide, **Prof. ANTONY S. ALEXANDER,**for his invaluable support and guidance throughout the development of this project. His encouragement, insights, and constructive feedback have greatly helped in shaping the project effectively. I also fondly remember and thank all my Professors of the Department of Computer Science and Applications, who helped me throughout my project journey.

I am truly thankful for their sociable approach, providing not only clear instructions but also fostering a collaborative and supportive environment. I am also grateful for the incredible support and opportunities that have played a crucial role in my successful completion of the project. I also thank the Loyola College Management, Department of Computer Science and Applications, for providing this wonderful opportunity for students so that they can gain knowledge and prepare themselves for the competitive world. I am truly thankful for their commitment to nurturing our academic and professional development.

LOURDU TYTUS GLASTIN R

TABLE OF CONTENTS

Chapter Number	Content	Page No.
1	INTRODUCTION	7
2	SYSTEM ANALYSIS	8
3	SYSTEM DESIGN	10
4	PROJECT DESCRIPTION	15
5	SYSTEM DEVELOPMENT	18
6	SYSTEM TESTING AND VALIDATION	24
7	USER MANUAL	34
8	SYSTEM DEVELOPMENT	38
9	CHAPTER 9: CONCLUSION	41
10	FUTURE ENHANCEMENTS	42
11	BIBLIOGRAPHY	43
	APPENDIX A - DATA DICTIONARY	44
	APPENDIX B - SAMPLE OUTPUT	45

CHAPTER 1
INTRODUCTION

The project "Authenticity – AI & Plagiarism Detection System" is a web-based application developed to analyze text and determine its originality. It combines plagiarism detection and AI-generated content identification in a single platform.

The system allows users to upload documents in PDF, DOCX, and TXT formats. The text is extracted and processed using techniques such as TF-IDF vectorization and cosine similarity to detect similarities. Additionally, statistical methods are used to estimate whether the content is AI-generated.

The system provides sentence-level similarity results, highlighted text, and visual reports such as charts and heatmaps. It also includes an admin dashboard for managing users and monitoring system activity. The project aims to provide a simple, efficient, and reliable solution for content authenticity verification.

Importance of the Project

The Authenticity Validator for Academia is important as it helps maintain academic integrity in an era where plagiarism and AI-generated content are increasing. It provides an automated way to detect copied and artificially generated text, reducing the need for manual checking. By offering accurate results and clear reports, the system supports fair evaluation and encourages originality among students, making it a valuable tool for modern educational institutions.

General Organization of the Report

This report presents the motivation and scope of the system, followed by an analysis of existing limitations and the proposed solution. It outlines the hardware and software requirements, describes the system architecture and design, and explains the development tools and implementation approach. The report further details the testing and validation procedures, along with the deployment process. Finally, it summarizes the outcomes of the system and discusses possible future enhancements.

CHAPTER 2
SYSTEM ANALYSIS

Problem Definition

The increasing availability of digital content creation tools and artificial intelligence systems has made it challenging to verify the authenticity of documents submitted in academic, research, and professional contexts. Traditional plagiarism detection methods often fail to identify sophisticated copying techniques or AI-generated content that mimics human writing patterns. Organizations require a reliable system that can analyze documents comprehensively, detecting both direct plagiarism through text similarity analysis and indirect indicators of artificial content generation. The current manual verification processes are time-consuming, subjective, and often ineffective against modern content manipulation techniques.

Existing System

Current document verification systems typically rely on manual review processes or basic text-matching algorithms. Many educational institutions use commercial plagiarism detection services that primarily focus on direct text matching against online databases. These systems often lack the capability to detect AI-generated content, which exhibits different characteristics than traditional plagiarism. Manual verification methods depend heavily on human expertise and are prone to inconsistencies. Existing tools are often expensive, require internet connectivity for database access, and may not support offline analysis of confidential documents. The limitations include inadequate detection of paraphrased content, inability to identify AI-generated text patterns, and lack of comprehensive reporting features for administrative oversight.

Proposed System

The proposed Document Authenticity Verification System offers a comprehensive solution that combines multiple analysis techniques within a user-friendly web application. The system enables users to upload documents in various formats and perform automated analysis for both plagiarism detection and AI content identification. Users can register accounts, manage

their documents, and access detailed analysis reports. Administrators can monitor system usage, manage user accounts, and view system-wide analytics. The system employs machine learning algorithms for text similarity analysis and heuristic methods for AI detection, providing quantitative scores and detailed reports. The web-based architecture ensures accessibility across different platforms while maintaining data security and user privacy.

System Requirements

Hardware Requirements

CPU	2 GHz dual- core of higher
RAM	Minimum 4GB (8GB recommended for heavy usage)
Storage	Minimum 500GB (SSD recommended for faster I/O)
OS	Windows 10/11
Network	Basic LAN/Wi-Fi for web access

Software Requirements

Python 3.8+
Flask (web framework)
SQLAlchemy (ORM)
SQLite (Database)
Scikit-learn (ML Algorithm)
NLTK (NLP preprocessing)
Numpy (numerical operations)
PyPDF2, python-docx (document parsing)
Matplotlib, Chart.js (Visualization)
Bootstrap 5 (frontend UI)
Werkzeug (utility functions)
Pip for dependency install, virtualenv/ venv for isolation

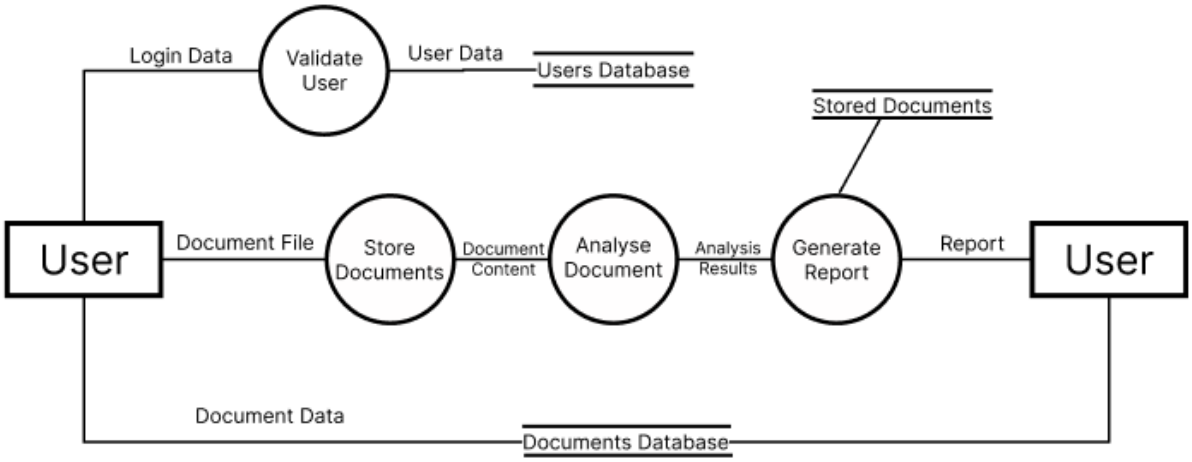
CHAPTER 3
SYSTEM DESIGN

Architectural Design

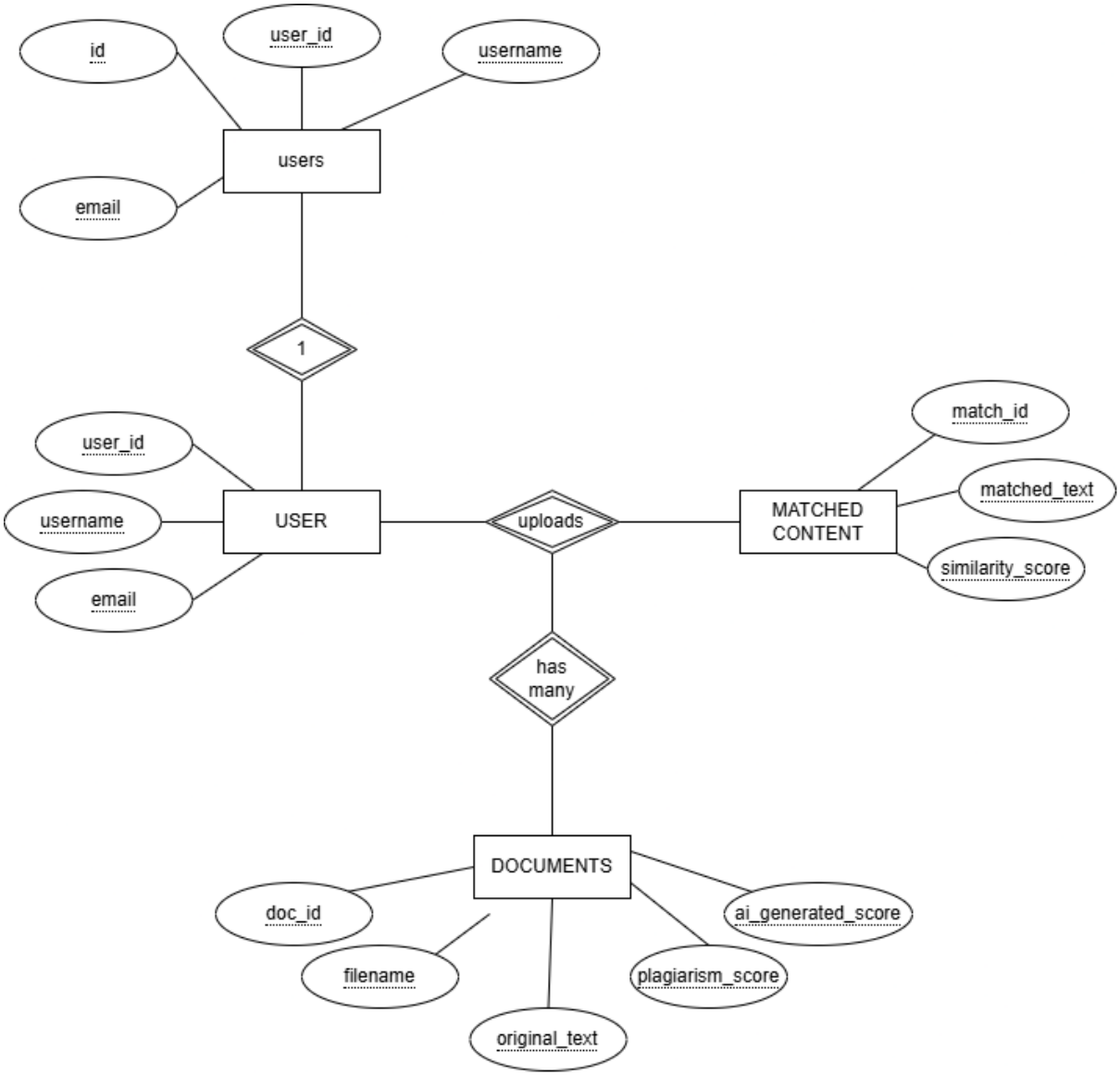
Data Flow Diagram

Level 0

Level 1:



Entity – Relationship Diagram:



Database Design

User Table

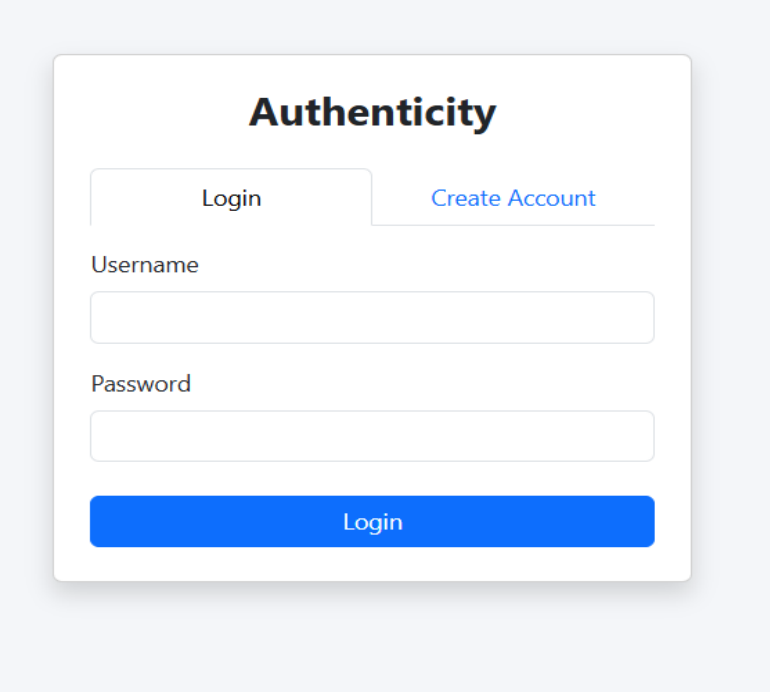
Field	Type	Constraints
id	INTEGER(11)	PRIMARY KEY, AUTOINCREMENT
username	VARCHAR(50)	UNIQUE, NOT NULL
email	VARCHAR(120)	UNIQUE, NOT NULL
password	VARCHAR(200)	NOT NULL
role	VARCHAR(20)	DEFAULT 'user'

Document Table

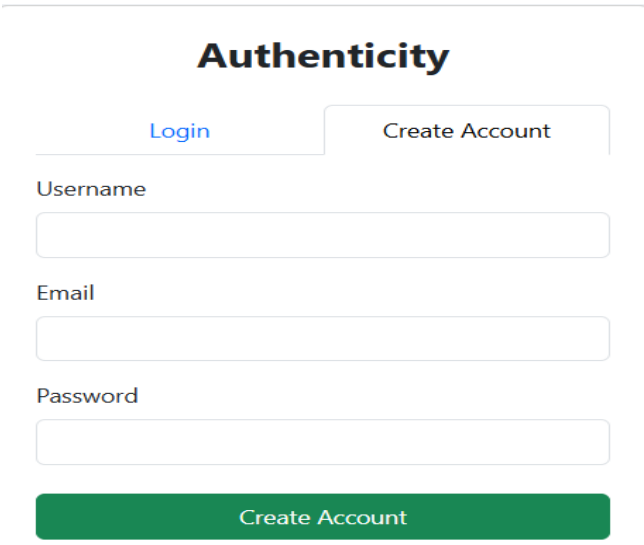
Field	Type	Constraints
id	INTEGER(11)	PRIMARY KEY, AUTOINCREMENT
user_id	INTEGER(11)	FOREIGN KEY REFERENCES user(id), NOT NULL
filename	VARCHAR(255)	NOT NULL
file_type	VARCHAR(10)	NOT NULL
upload_time	DATETIME	DEFAULT CURRENT_TIMESTAMP
original_text	TEXT	
plagiarism_score	FLOAT	DEFAULT 0.0
ai_generated_prob	FLOAT	DEFAULT 0.0
similarity_report	TEXT	

GUI Design

Login page:



Create Account:



Upload Document page:

Upload Document

Choose File ai.pdf

Upload

Dashboard page:

Uploaded Documents

Upload Document

#	Filename	Plagiarism	AI Probability	Text Length	Action
1	ai.pdf	0.00%	48.88%	783	View
2	human.pdf	0.00%	0.00%	141	View
3	humanized_content.pdf	0.00%	51.90%	473	View
4	ai.pdf	100.00%	48.88%	783	View

Summary Page:

ai.pdf

Plagiarism Score: 100.0%

AI Probability: 48.88%

Matched Sentences

Sentence	Source Document	Similarity
artificial intelligence systems are computational frameworks designed to simulate aspects of human cognition through structured algorithms and datadriven models	ai.pdf	100.0%
these systems operate by analyzing large volumes of input data identifying patterns and generating outputs based on learned relationships rather than explicit programming for each task	ai.pdf	100.0%
modern ai architectures particularly those utilizing machine learning and deep learning techniques rely on iterative training processes to optimize performance metrics and improve predictive accuracy over time	ai.pdf	100.0%
as a result ai is capable of performing complex tasks such as natural language processing image recognition and decision making with a high degree of efficiency consistency and scalability across diverse application domains	ai.pdf	100.0%

CHAPTER 4
Project Description

The Authenticity Validator for Academia is a comprehensive web-based system designed to ensure the originality and integrity of academic documents through automated analysis. The system is built using a modular architecture that separates functionalities into distinct components, allowing for better organization, scalability, and maintainability. Each module performs a specific role while interacting seamlessly with others, enabling efficient processing and analysis of uploaded documents. The overall design follows a layered approach, where the frontend manages user interaction, the backend handles processing logic, and the database stores and retrieves information. This structure not only simplifies development but also makes the system adaptable to future technological enhancements.

At the core of the system lies the main application module, which acts as the central control unit. This module initializes the application, configures essential settings such as secret keys, database connections, and file storage directories, and integrates necessary extensions for database management and authentication. It defines the primary routes that govern the flow of the application, including user registration, login, document upload, analysis processing, dashboard display, and administrative functionalities. By organizing these routes effectively, the system ensures smooth navigation and a user-friendly experience. The centralized control also enables efficient coordination between different modules, ensuring that all operations are performed reliably.

Security and access control are key aspects of the system, especially considering the sensitive nature of academic data. The application implements secure authentication mechanisms by storing user passwords in hashed form and verifying them during login. Session management is used to maintain user state, ensuring that only authenticated users can access specific features. Additionally, role-based authorization is incorporated to distinguish between regular users and administrators. While users can upload and analyze documents, administrators have access to advanced functionalities such as monitoring system usage, viewing aggregated analytics, and managing user accounts. This structured access control enhances both security and system reliability.

The system utilizes a well-defined database structure to manage data efficiently. It includes separate entities for users and documents, with a clear relationship established between them. Each user can upload multiple documents, creating a one-to-many relationship that allows for organized storage and retrieval. Document records contain detailed information, including file metadata, extracted text, plagiarism scores, AI-generated probability, and similarity reports. This structured approach ensures that all relevant information is preserved and easily accessible for future reference, analysis, or auditing purposes.

A crucial component of the application is the text extraction module, which enables the system to process documents in various formats such as TXT, PDF, and DOCX. The module intelligently selects the appropriate method for extracting text based on the file type, ensuring compatibility and flexibility. Once the text is extracted, it undergoes preprocessing to remove unnecessary characters, normalize formatting, and standardize the content. This step is essential for improving the quality of the data and ensuring accurate analysis, as it eliminates inconsistencies that may affect the performance of the detection algorithms.

The plagiarism detection functionality is implemented using advanced Natural Language Processing techniques. The system converts textual data into numerical representations using TF-IDF vectorization and then applies cosine similarity to measure the degree of similarity between documents. This approach allows the system to detect both exact matches and paraphrased content effectively. Furthermore, the system performs sentence-level analysis to provide more detailed insights, highlighting specific portions of text that may have been copied. This granular analysis enhances the accuracy and usefulness of the results, making it easier for users to identify problematic sections.

In addition to plagiarism detection, the system incorporates an AI content detection module to address the growing use of artificial intelligence in content creation. This module analyzes linguistic features such as lexical diversity, repetition patterns, sentence uniformity, and average sentence length to estimate the likelihood of the content being generated by AI. By combining these factors using a weighted scoring method, the system produces a probability value that indicates the presence of AI-generated text. This feature adds significant value to the application by tackling a modern challenge that is not addressed by traditional plagiarism detection tools.

The analytics component further enhances the system by providing visual representations of the analysis results. Charts and graphical outputs are generated to help users and administrators understand patterns and trends in document submissions. These visual insights make the system more interactive and user-friendly, enabling better interpretation of complex data. Administrators, in particular, benefit from these analytics as they can monitor overall system performance and identify areas that require attention.

Overall, the Authenticity Validator for Academia represents a well-integrated solution that combines multiple technologies to address the problem of academic dishonesty. By integrating text processing, machine learning techniques, secure authentication, and data visualization, the system provides a reliable and efficient platform for verifying document authenticity. Its modular design ensures scalability and flexibility, making it suitable for real-world academic environments. The system not only simplifies the process of document verification but also promotes fairness, transparency, and originality in academic practices, thereby contributing positively to the educational ecosystem.

CHAPTER 5

System Development

Language / Tool

The development of the Authenticity Validator for Academia utilizes a combination of modern programming languages and tools to ensure efficiency, scalability, and ease of maintenance. The backend of the system is developed using Python, which is widely preferred for its simplicity, readability, and extensive support for libraries related to data processing and machine learning. The Flask framework is used as the core web framework, providing lightweight and flexible routing, request handling, and session management capabilities. SQLAlchemy is employed as the Object Relational Mapper (ORM), allowing seamless interaction with the SQLite database by mapping database tables to Python classes. SQLite is chosen as the database due to its lightweight nature and suitability for small to medium-scale applications, ensuring efficient storage and retrieval of user and document data.

For security and authentication, Werkzeug is used to implement password hashing and secure file handling, while Flask-JWT-Extended enables token-based authentication and authorization mechanisms. The system also integrates libraries such as scikit-learn for implementing machine learning algorithms, particularly TF-IDF vectorization and cosine similarity for plagiarism detection. NLTK is used for natural language preprocessing tasks such as tokenization and text normalization, while NumPy supports numerical computations. Additional libraries such as PyPDF2 and python-docx are utilized for extracting text from PDF and DOCX files, ensuring compatibility with multiple document formats.

On the frontend, the system is developed using HTML and CSS, with Bootstrap providing responsive design and user-friendly interface components. JavaScript is used to enhance interactivity, while Chart.js is integrated to display analytical results in the form of charts and visual representations. The overall integration of these tools ensures that the system delivers a smooth user experience, robust backend processing, and effective data visualization, making it a comprehensive solution for academic document verification.

Pseudo code

Application Initialization

```
BEGIN
   Initialize Flask Application
   Set SECRET_KEY and DATABASE_URI
   Configure UPLOAD_FOLDER
   Initialize Database (SQLAlchemy)
   Initialize Authentication (JWT)
   Create required folders if not exist
   Create database tables
END
```

User Registration

```
BEGIN
   INPUT username, email, password
   IF username OR email already exists in database THEN
      DISPLAY "User already exists"
   ELSE
      HASH password
      CREATE new user record
      STORE user in database
      DISPLAY "Registration successful"
   ENDIF
END
```

User Login

```
BEGIN
   INPUT username, password

   FETCH user from database WHERE username = input

   IF user exists THEN
      IF password matches hashed password THEN
```

```
        CREATE user session
        IF role = admin THEN
            REDIRECT to admin dashboard
        ELSE
            REDIRECT to user dashboard
        ENDIF
    ELSE
        DISPLAY "Invalid password"
    ENDIF
  ELSE
    DISPLAY "User not found"
  ENDIF
END
```

Document Upload and Processing

```
BEGIN
  VERIFY user session

  INPUT file

  IF file is not empty AND format is valid THEN
    SAVE file to upload folder
    EXTRACT text from file
    PREPROCESS text (clean, normalize)

    CALL plagiarism_detection(text)
    CALL ai_detection(text)

    STORE file details and results in database
    DISPLAY "Upload and analysis successful"
  ELSE
    DISPLAY "Invalid file"
  ENDIF
END
```

Plagiarism Detection

```
FUNCTION plagiarism_detection(new_text)

    FETCH all existing documents

    APPLY TF-IDF vectorization on new_text and existing documents

    COMPUTE cosine similarity

    FOR each document
        CALCULATE similarity score
        STORE matching segments
    ENDFOR

    RETURN maximum similarity score and report
END FUNCTION
```

AI Content Detection

```
FUNCTION ai_detection(text)

    CALCULATE lexical diversity
    CALCULATE repetition score
    CALCULATE sentence uniformity
    CALCULATE average sentence length

    APPLY weighted formula:
        score = (0.30 * diversity) +
                (0.25 * repetition) +
                (0.25 * uniformity) +
                (0.20 * length)

    LIMIT score between 0 and 100
```

```
    RETURN AI probability score
END FUNCTION
```

Store Results

```
BEGIN
    CREATE new document record

    STORE:
        user_id
        filename
        extracted text
        plagiarism score
        AI probability
        similarity report

    SAVE record in database
END
```

Dashboard Display

```
BEGIN
    VERIFY user session

    FETCH all documents of user

    DISPLAY:
        document list
        plagiarism scores
        AI scores
        summary charts
END
```

Admin Analytics

```
BEGIN
    VERIFY admin role
```

```
    FETCH all users and documents

    CALCULATE:
        total users
        total documents
        average plagiarism score

    GENERATE charts

    DISPLAY analytics dashboard
END
```

Validation and Error Handling

```
BEGIN
    CHECK file format and size
    CHECK extracted text is not empty
    VERIFY user authentication for protected routes

    IF error occurs THEN
        DISPLAY user-friendly error message
        LOG error for debugging
    ENDIF
END
```

CHAPTER 6
System Testing and Validations

System testing is an essential phase in the development of the Authenticity Validator for Academia, ensuring that the system performs accurately, reliably, and efficiently under different conditions. The testing process was carried out systematically to validate both individual components and the integrated system as a whole. Various types of testing, including unit testing, integration testing, acceptance testing, and validation testing, were conducted to identify errors, verify functionality, and ensure that the system meets user requirements. Each type of testing focuses on a specific aspect of the system, such as individual module correctness, interaction between components, user satisfaction, and input validation. The results obtained from testing confirm that the system operates as intended and provides accurate outputs for different scenarios.

Unit Testing

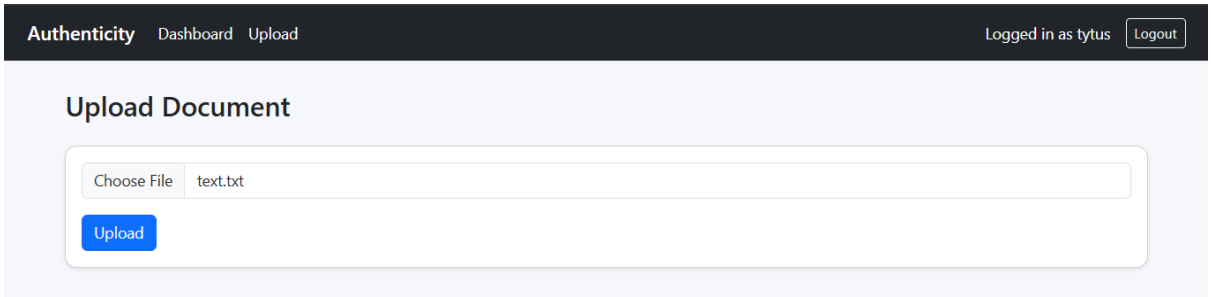
Unit testing focuses on verifying the correctness of individual modules such as text extraction, preprocessing, plagiarism detection, and AI content detection. Each module is tested independently by providing controlled inputs and checking whether the output matches the expected result. This helps in identifying and fixing errors at an early stage of development.

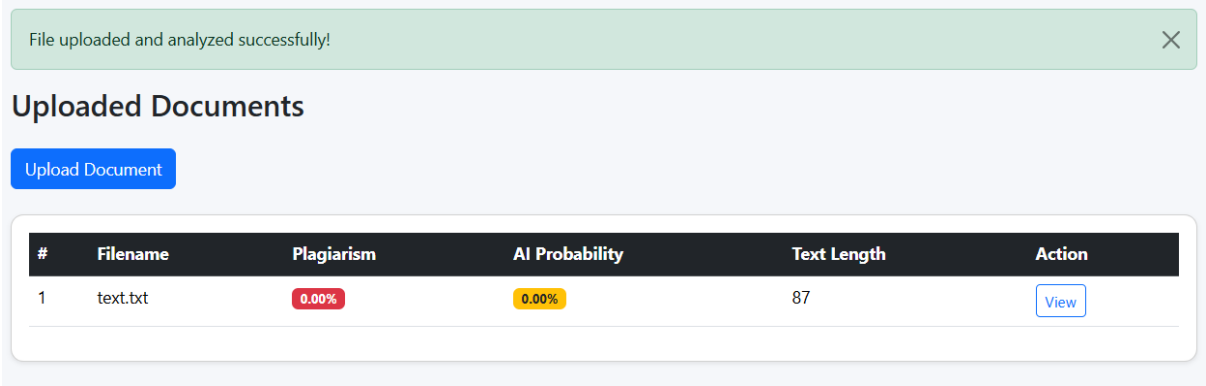
Test Case 1: Valid File Text Extraction

Input:
Upload a valid .txt or .pdf file containing readable text.

Expected Output:
The system should successfully extract text from the file and display the document in the dashboard without any errors.

Actual Output:
The system successfully extracted the text, processed the file, and displayed the document along with analysis results in the dashboard.





Test Case 2: Invalid File Handling

Input:

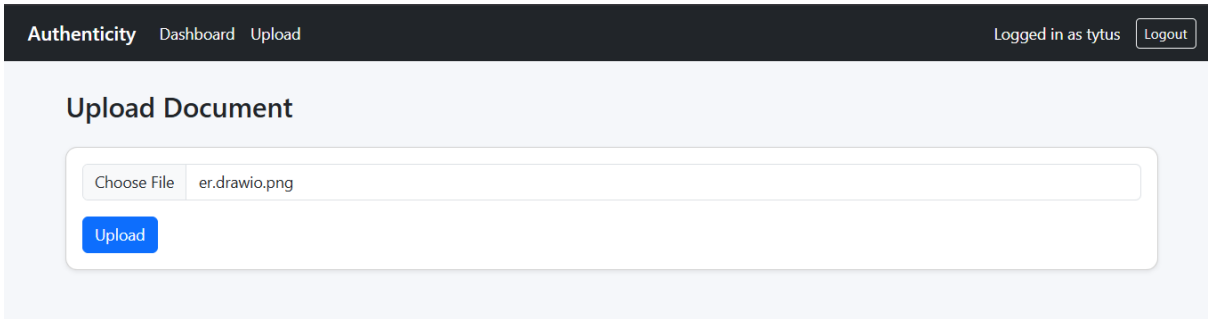
Upload an empty file or unsupported format.

Expected Output:

The system should reject the file and display an appropriate error message.

Actual Output:

The system displayed an error message indicating invalid file input and did not process the file.



No readable text found

Integration Testing

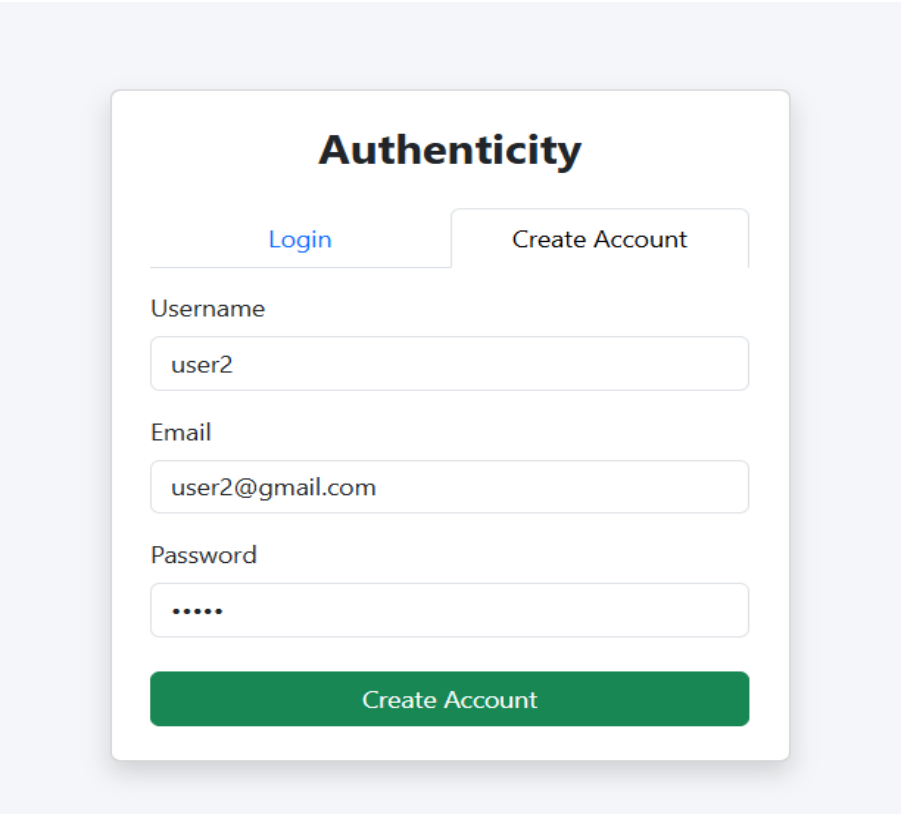
Integration testing ensures that different modules of the system work together correctly. It verifies the flow of data between components such as user authentication, file upload, text processing, and result generation. This testing confirms that the system functions smoothly as a complete unit.

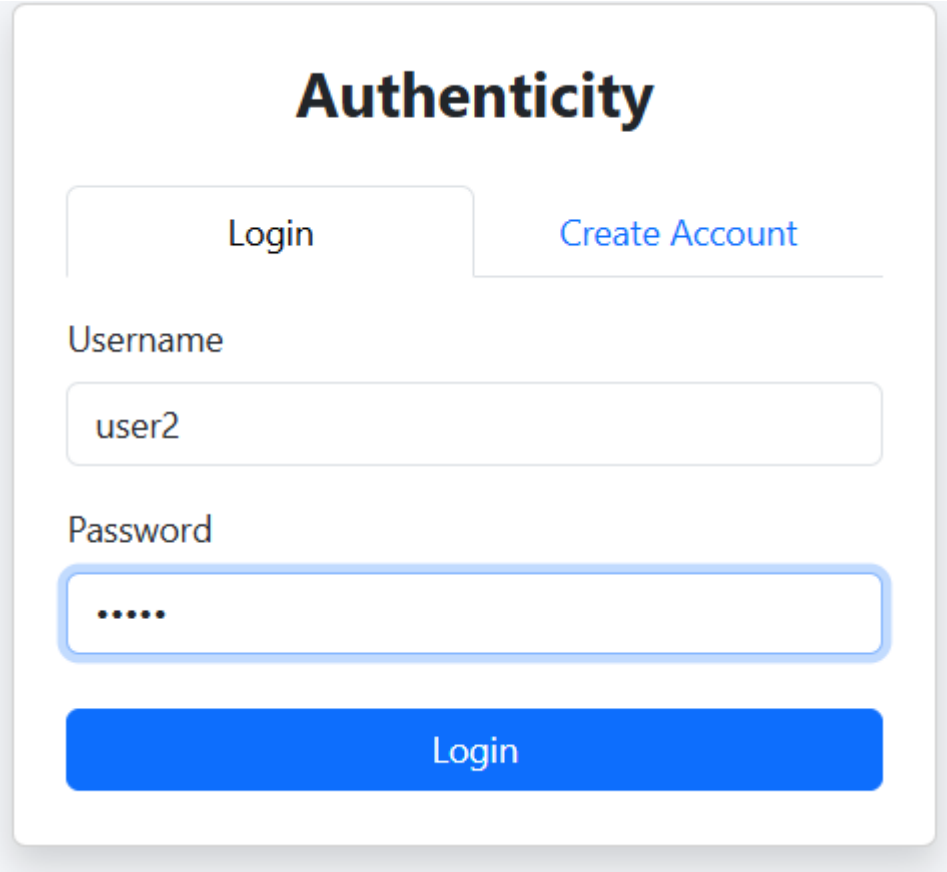
Test Case 1: Complete Workflow Execution

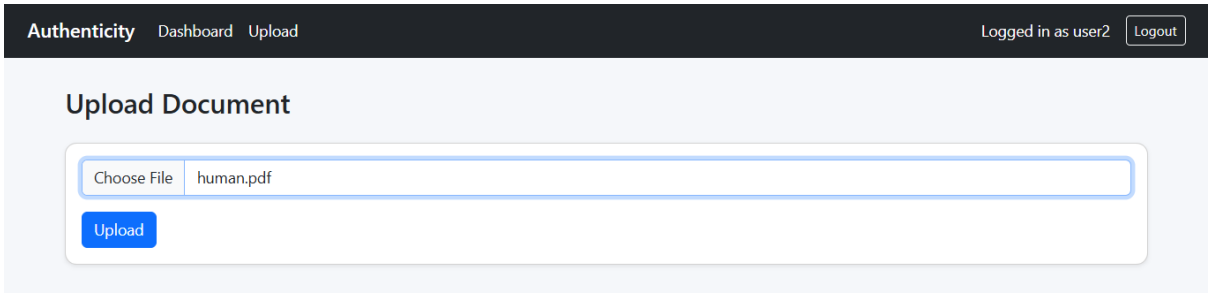
Input:
Register a user, login, upload a document, and view results.

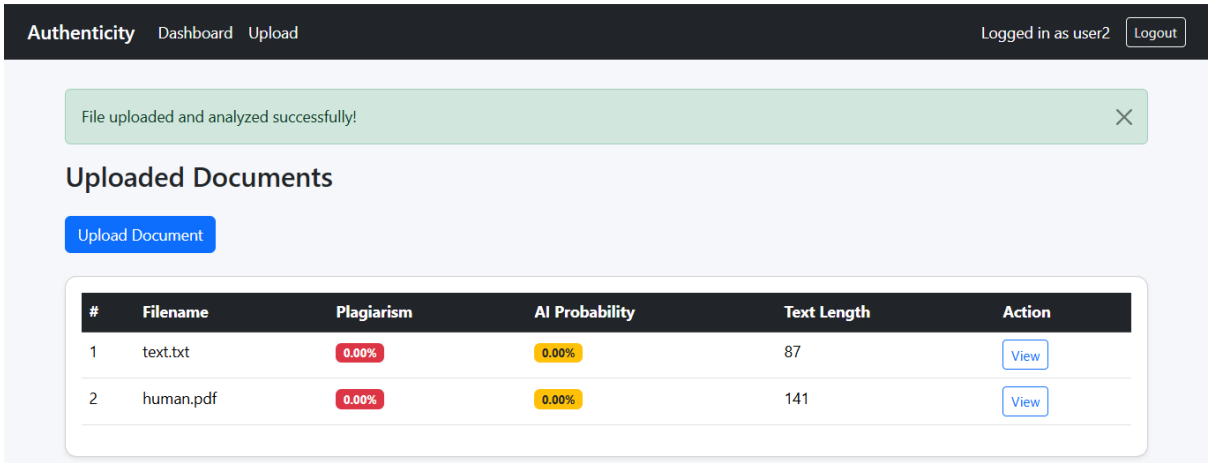
Expected Output:
The system should allow successful login, process the uploaded document, and display plagiarism and AI detection results.

Actual Output:
The system successfully completed the workflow, allowing login, processing the document, and displaying accurate results on the dashboard.


Authenticity
Login
Create Account
Username
user2
Email
user2@gmail.com
Password
•••••
Create Account


Authenticity
Login
Create Account
Username
user2
Password
•••••
Login





Test Case 2: Data Storage Verification

Input:

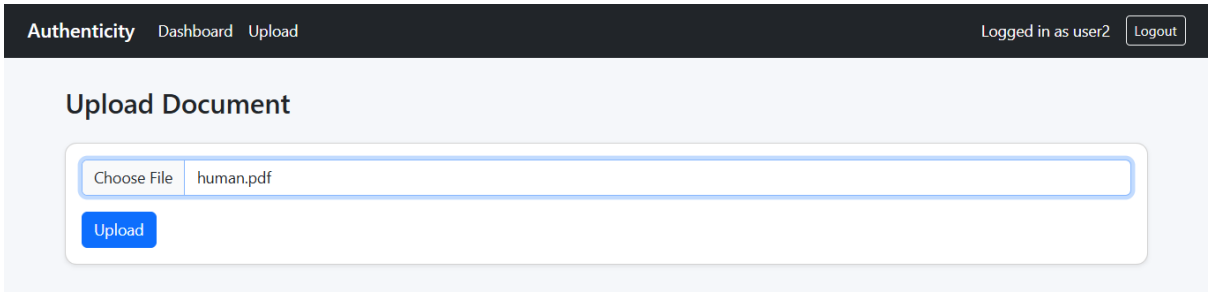
Upload a document and check stored data.

Expected Output:

The document details and analysis results should be stored correctly in the database.

Actual Output:

The system stored all document details, including filename, plagiarism score, and AI probability, correctly in the database.



human.pdf

Plagiarism Score: 100.0%

AI Probability: 0.0%

Matched Sentences

Sentence	Source Document	Similarity
hi this is tytus and im from chennai	human.pdf	100.0%
i study bsc computer science in loyola college	human.pdf	100.0%
i finished my school in kendirya vidayalaya iit campus	human.pdf	100.0%

Acceptance Testing

Acceptance testing is performed to evaluate whether the system meets the requirements of end users and functions effectively in real-world scenarios. It focuses on validating user interactions, ease of use, and overall system behavior. This testing ensures that the system provides a smooth and reliable experience for users while performing key operations such as uploading documents and viewing analysis results.

Test Case 1: User Document Upload and Analysis

Input:

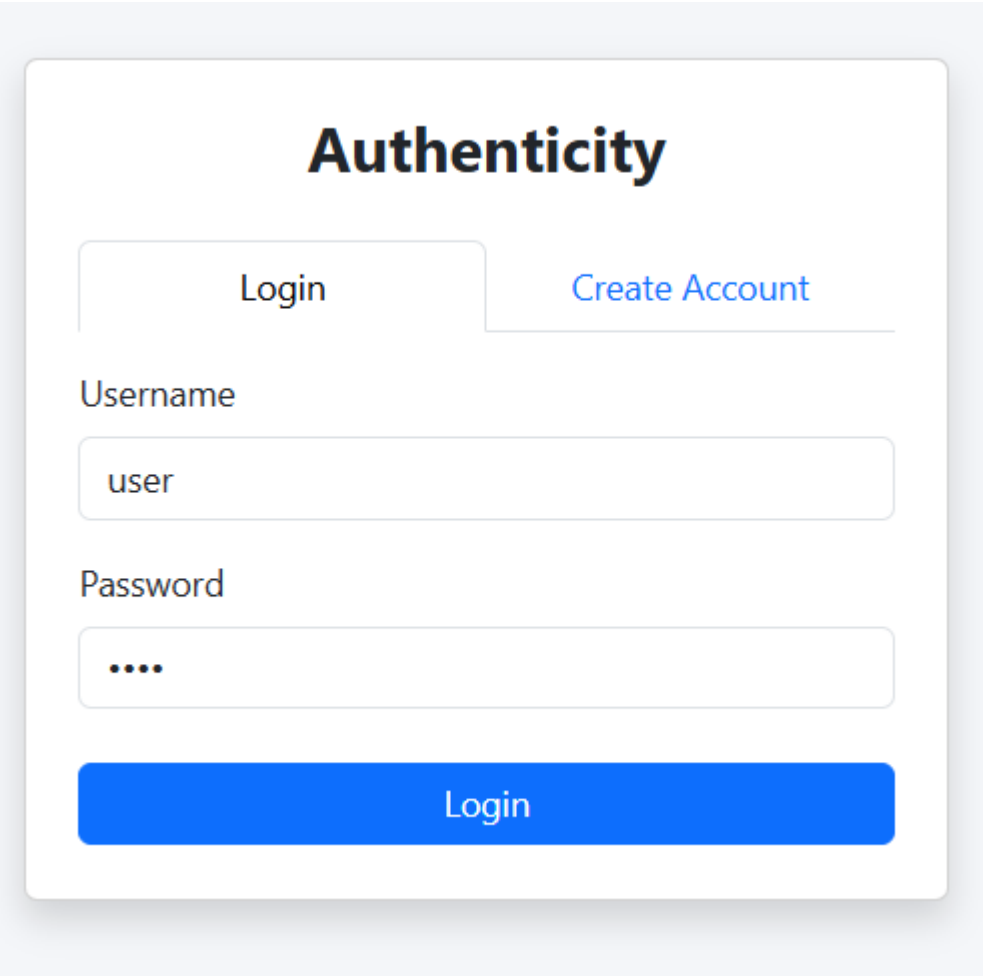
Login as a user and upload a valid document file.

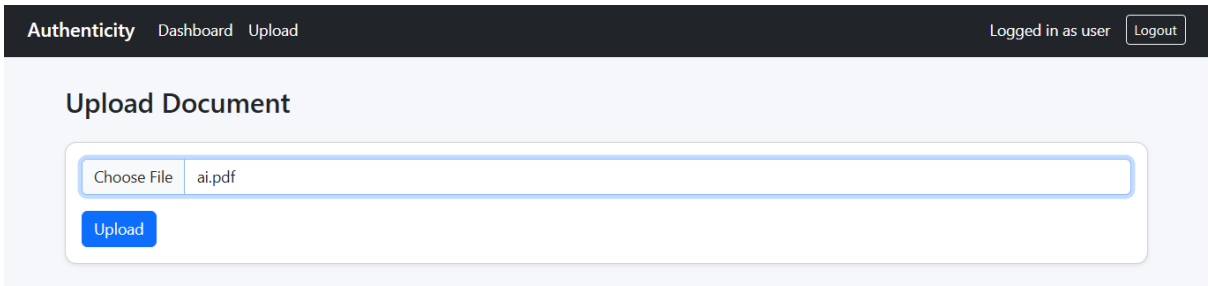
Expected Output:

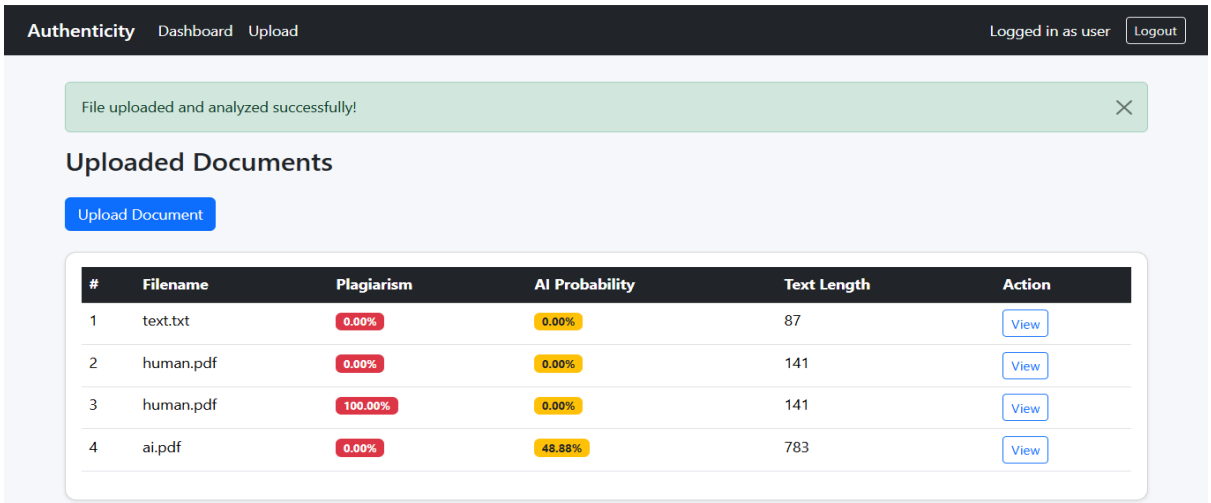
The system should successfully upload the document, process it, and display plagiarism score and AI probability.

Actual Output:

The system successfully uploaded the document, processed it, and displayed accurate plagiarism and AI detection results.


Authenticity
Login
Create Account
Username
user
Password
••••
Login


Authenticity
Dashboard
Upload
Logged in as user
Logout
Upload Document
Choose File
ai.pdf
Upload


Authenticity
Dashboard
Upload
Logged in as user
Logout
File uploaded and analyzed successfully!
Uploaded Documents
Upload Document
Filename Plagiarism AI Probability Text Length Action
1 text.txt 0.00% 0.00% 87 View
2 human.pdf 0.00% 0.00% 141 View
3 human.pdf 100.00% 0.00% 141 View
4 ai.pdf 0.00% 48.88% 783 View

Test Case 2: Viewing Document Results

Input:

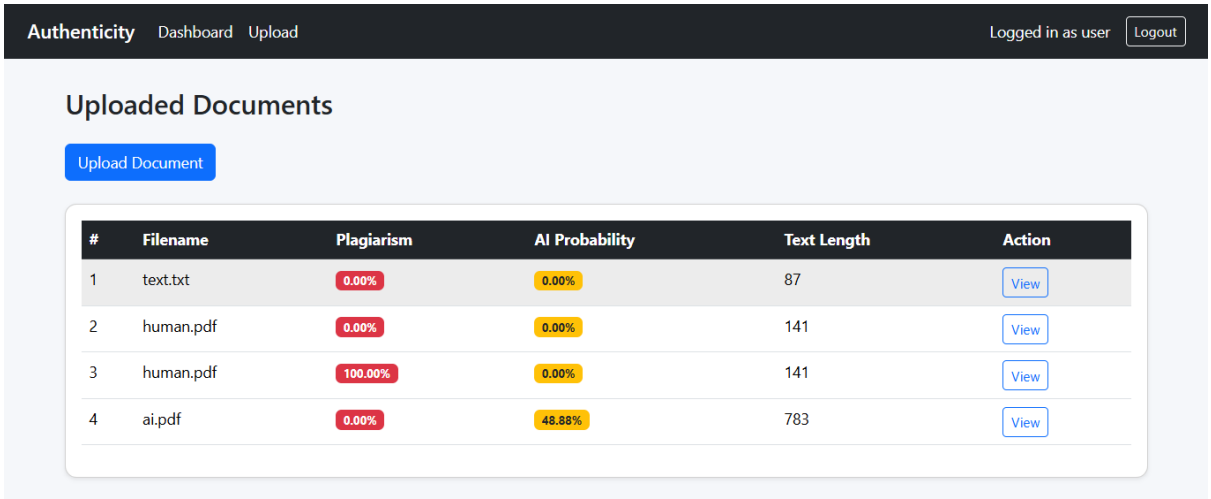
Select a previously uploaded document from the dashboard.

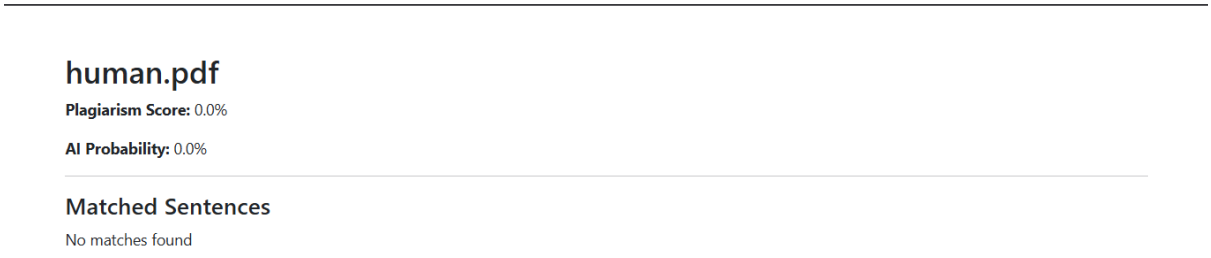
Expected Output:

The system should display detailed results including similarity score, AI probability, and analysis report.

Actual Output:

The system displayed complete document details along with analysis results correctly without any errors.





6.4 Validation Testing

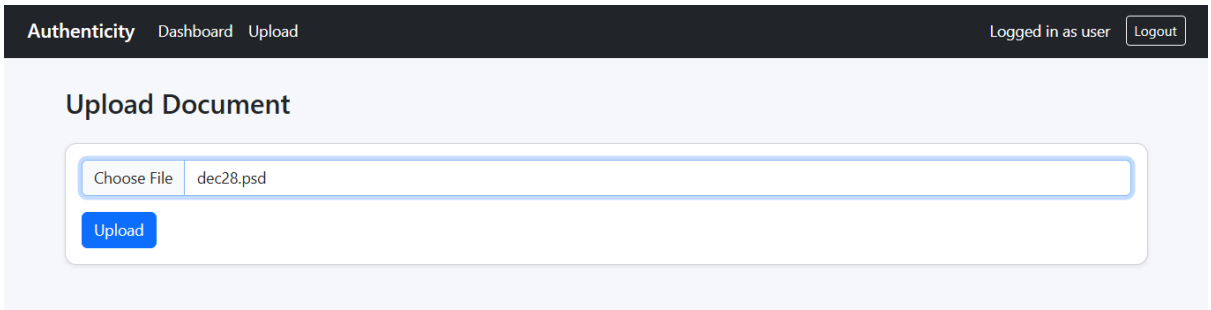
Validation testing ensures that the system handles incorrect inputs and maintains data integrity. It verifies that proper validation checks are implemented for inputs such as file uploads and user authentication.

Test Case 1: File Format Validation

Input:
Upload an unsupported file format (e.g., .exe).

Expected Output:
The system should reject the file and display an error message.

Actual Output:
The system successfully rejected the file and displayed an appropriate error message.



No readable text found

Test Case 2: Invalid Login Credentials

Input:

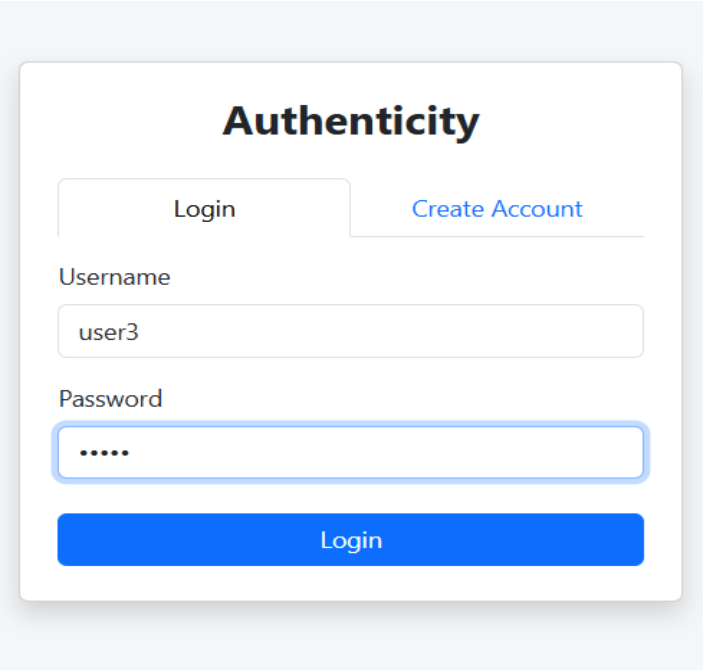
Enter incorrect username or password.

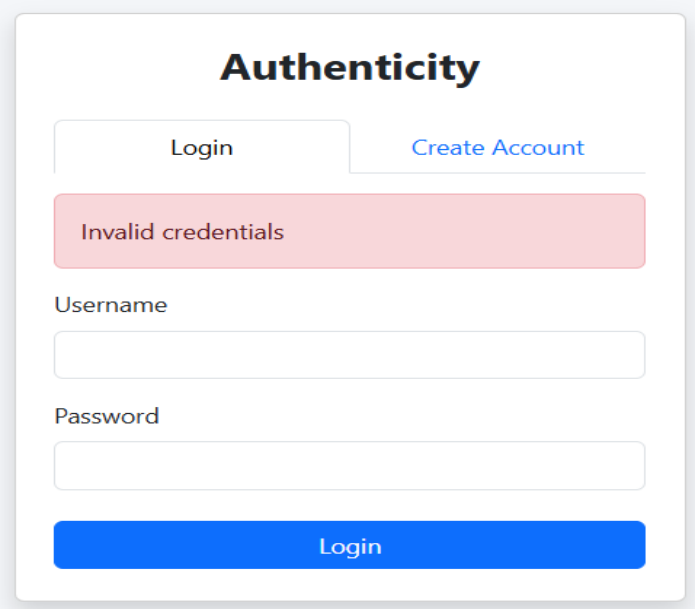
Expected Output:

The system should deny access and display an error message.

Actual Output:

The system displayed an error message and prevented login access.





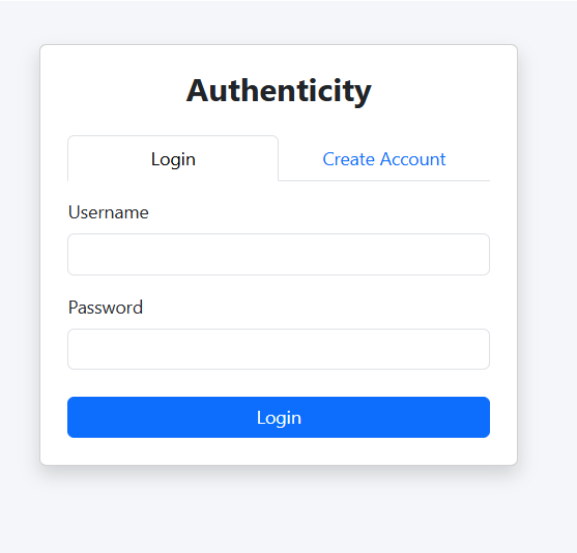
CHAPTER 7

User Manual

The user manual provides comprehensive instructions for using the Authenticity platform. Each major function is covered from registration through document analysis, including sample screenshots labels that can be replaced with actual images in final submission.

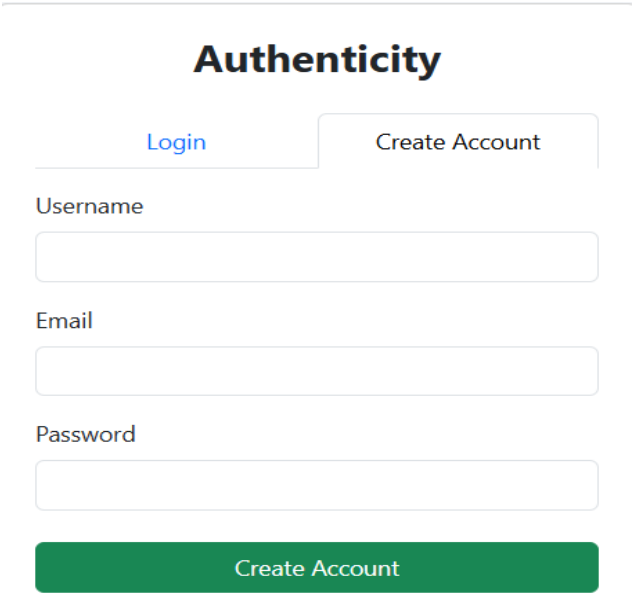
Running the Application

1. Start server:
   - `python app.py`
2. Open browser:
   - `http://127.0.0.1:5000/`
3. Expected landing page: login screen.



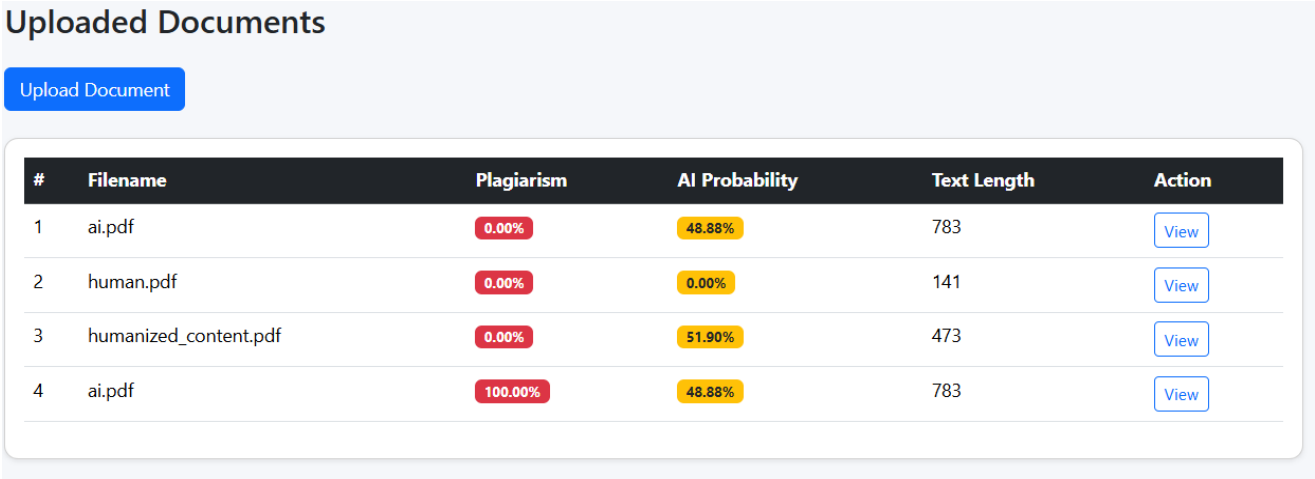
User Registration

1. Click 'Register' (or open `/create_user`).
2. Enter username, email, password.
3. Submit form.
4. Successful registration redirects to login.



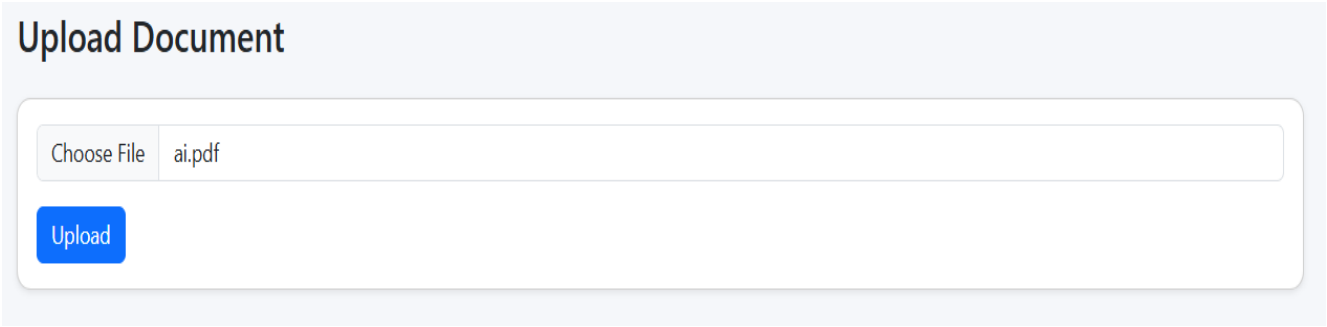
User Login

1. Enter username and password.
2. Submit form.
3. Successful login based on role: user -> `/dashboard`; admin -> `/admin`.



Upload Document

1. On `/upload`, choose a file (`.txt`, `.pdf`, `.docx`).
2. Click upload.
3. System processes file, calculates plagiarism and AI scores.
4. On success, redirects to dashboard and shows message.



Viewing Document Details

1. From dashboard, click a document entry.
2. Open detail view to inspect:
   - file metadata
   - plagiarism score
   - AI probability
   - similarity report list

ai.pdf

Plagiarism Score: 100.0%

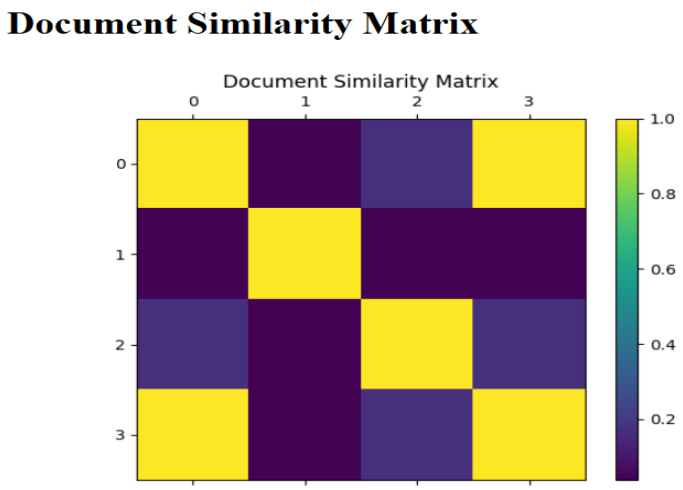
AI Probability: 48.88%

Matched Sentences

Sentence	Source Document	Similarity
artificial intelligence systems are computational frameworks designed to simulate aspects of human cognition through structured algorithms and datadriven models	ai.pdf	100.0%
these systems operate by analyzing large volumes of input data identifying patterns and generating outputs based on learned relationships rather than explicit programming for each task	ai.pdf	100.0%
modern ai architectures particularly those utilizing machine learning and deep learning techniques rely on iterative training processes to optimize performance metrics and improve predictive accuracy over time	ai.pdf	100.0%
as a result ai is capable of performing complex tasks such as natural language processing image recognition and decision making with a high degree of efficiency consistency and scalability across diverse application domains	ai.pdf	100.0%

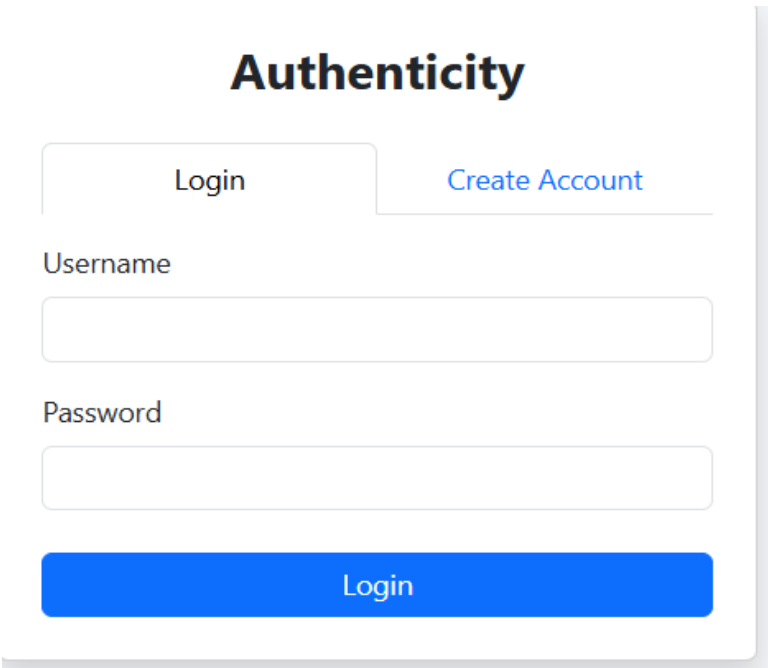
Similarity Graph

1. Navigate to `/similarity-graph`.
2. View heatmap of cosine similarity between documents.



Logout

1. Click logout button.
2. Session is cleared and redirects to login.



CHAPTER 8
System Deployment

Local Deployment

System deployment is the process of making the Authenticity Validator for Academia application available for use in a real-world environment. It involves setting up the required hardware and software, installing dependencies, configuring the database, and hosting the application either on a standalone system or a server. Proper deployment ensures that the system runs efficiently, securely, and is accessible to users without errors. This section describes the requirements and step-by-step procedure followed to install, configure, and run the application.

Installation and Setup

1. Clone repository:
   - `git clone <repo-url>`
2. Create and activate environment:
   - `python -m venv venv`
   - On Windows: `venv\Scripts\activate`
3. Install dependencies:
   - `pip install -r requirements.txt`
4. Create required folders:
   - `mkdir instance uploads static\charts`
5. Verify files:
   - `app.py`, `models/`, `utils/`, `templates/` exist.

Deployment Environment and Requirements

The deployment of the system requires both hardware and software components to function effectively. The hardware requirements include a system with at least a dual-core processor, a minimum of 4 GB RAM, and sufficient storage space for handling uploaded documents and database files. A stable operating system such as Windows 10/11 or Linux is required for

compatibility. On the software side, Python 3.8 or higher must be installed along with necessary libraries such as Flask, SQLAlchemy, scikit-learn, NLTK, NumPy, and other dependencies specified in the requirements file. Additionally, tools such as pip for package management and a virtual environment are used to isolate project dependencies. These requirements ensure that the application runs smoothly without conflicts or performance issues.

Installation of the Application

The installation process begins by setting up the project environment on the system. First, the project source code is obtained either by cloning the repository or copying the project folder. A virtual environment is then created using Python to maintain dependency isolation. After activating the virtual environment, all required libraries are installed using the requirements file. Necessary folders such as uploads, instance, and static/charts are created to store files and generated outputs. Once the dependencies are installed and the folder structure is ready, the application is initialized, and the database tables are created automatically using the configured ORM. This completes the installation process and prepares the system for execution.

Database Setup and Configuration

The system uses SQLite as the database for storing user information and document data. During the initial setup, the database is automatically created when the application is run for the first time. The database schema is defined using SQLAlchemy models, which ensures proper structure and relationships between tables such as users and documents. The database stores important details including user credentials, uploaded file information, plagiarism scores, and AI-generated probability results. Proper configuration of the database ensures data integrity, efficient retrieval, and secure storage of information required for system operations.

Running the Application (Standalone System)

After installation and configuration, the application can be executed on a standalone system. The user starts the application by running the main Python file, which launches the Flask development server. Once the server is running, the application can be accessed through a web browser using the local host address. The login page is displayed, allowing users to register and access the system. The standalone deployment is suitable for development, testing, and

demonstration purposes, as it provides a simple and controlled environment for running the application.

Web Application Hosting (Server Deployment)

For broader accessibility, the application can be deployed on a server. In a production environment, the Flask application is typically run using a WSGI server such as Gunicorn, which improves performance and handles multiple requests efficiently. A reverse proxy server such as Nginx can be configured to manage incoming traffic, handle static files, and provide secure communication through HTTPS. Environment variables are used to securely store sensitive information such as secret keys and database configurations. This type of deployment ensures that the application is accessible to multiple users over a network while maintaining stability and security.

Activation and Usage of the System

Once the system is deployed, users can access the application through a web browser. New users can create accounts, log in, and upload documents for analysis. The system processes the uploaded files and displays results such as plagiarism scores and AI probability. Users can view their document history and analysis reports through the dashboard. The activation process is straightforward, requiring only a running server and a browser, making the system user-friendly and easy to operate.

Maintenance and Updates

Regular maintenance is essential to ensure the smooth functioning of the system. This includes updating software dependencies, monitoring system performance, and managing stored data. Backup of the database should be performed periodically to prevent data loss. In addition, new features and improvements can be added to the system as part of future updates. Proper maintenance ensures that the application remains reliable, secure, and efficient over time.

CHAPTER 9
Conclusion

The Authenticity Validator for Academia successfully addresses the growing need for reliable and efficient verification of academic documents in an era increasingly influenced by digital content and artificial intelligence. The system integrates plagiarism detection and AI-generated content identification into a single, unified platform, providing users with a comprehensive solution for evaluating the originality of textual submissions. By utilizing techniques such as TF-IDF vectorization and cosine similarity, the system is capable of detecting both direct copying and paraphrased content with a high degree of accuracy. In addition, the inclusion of AI detection mechanisms based on linguistic analysis enhances the system's capability to identify artificially generated text, which is becoming a significant concern in academic environments. The application offers a user-friendly interface that allows users to upload documents, view analysis results, and access detailed reports with ease. Features such as sentence-level similarity detection and visual representations further improve the interpretability of results, making the system both practical and informative. From a technical perspective, the use of Python and Flask ensures a robust backend, while tools such as SQLAlchemy and SQLite provide efficient data management. The modular design of the system allows for scalability and future enhancements without disrupting existing functionalities. Throughout the development process, systematic testing and validation ensured that the system performs reliably under various conditions. Overall, the project demonstrates a successful implementation of modern technologies to solve a real-world problem, contributing to the promotion of academic integrity, fairness, and transparency. It provides a strong foundation for further development and can be effectively utilized in educational institutions to streamline the process of document verification and uphold ethical academic practices.

CHAPTER 10
Future Enhancement

While the Authenticity Validator for Academia provides a robust solution for detecting plagiarism and AI-generated content, there are several opportunities for further enhancement to improve its functionality and scalability. One of the key future improvements includes the integration of Optical Character Recognition (OCR) technology, which would enable the system to process scanned documents and images containing text, thereby expanding its usability. Additionally, incorporating advanced machine learning models such as transformer-based architectures can significantly enhance the accuracy of both plagiarism detection and AI content identification by capturing deeper semantic relationships within the text. The system can also be extended to support multiple languages, making it more accessible to a wider range of users across different regions. Another important enhancement is the development of a RESTful API, which would allow seamless integration with Learning Management Systems (LMS) and other academic platforms, enabling automated submission and evaluation processes. Improvements in user management, such as more granular role-based access control and audit logs, can enhance system security and accountability. The addition of real-time notifications and alerts for high plagiarism or AI-generated content can further assist educators in making timely decisions. From a deployment perspective, migrating the system to a cloud-based infrastructure can improve scalability, availability, and performance for handling larger volumes of data. Furthermore, implementing continuous model training using user feedback can help the system adapt to evolving patterns in content generation. The inclusion of detailed feedback and suggestions for users to improve their writing can also transform the system into a learning tool rather than just a detection mechanism. These enhancements will not only improve the overall efficiency and accuracy of the system but also ensure its relevance and effectiveness in addressing future challenges in academic integrity.

CHAPTER 11
Bibliography

The bibliography references official documentation and academic resources that guided system design and implementation. Key references used include:

- Flask official documentation (https://flask.palletsprojects.com/)
- SQLAlchemy documentation (https://docs.sqlalchemy.org/)
- scikit-learn user guide TF-IDF and cosine similarity (https://scikit-learn.org/stable/modules/feature_extraction.html)
- pdfplumber docs (https://github.com/jsvine/pdfplumber)
- python-docx docs (https://python-docx.readthedocs.io/)
- Flask-JWT-Extended docs (https://flask-jwt-extended.readthedocs.io/)
- matplotlib plotting docs (https://matplotlib.org/stable/contents.html)
- NLTK and spaCy preprocessing best practices for text normalization in Python
- Research: "A survey of machine learning for big code and naturalness" (Allamanis et al., 2018)
- Research: "GLTR: Statistical detection and visualization of generated text" (Gehrmann et al., 2019)
- Research: "A robust plagiarism detection system for educational usage" (Crowe et al., 2017)
- Research: "Legal and ethical aspects of AI-generated content in education" (Wagner & Reinsel, 2022)

Appendix A - Data Dictionary

User table

Column	Type	Description
id	Integer	Primary key (auto-increment)
username	String	Unique login identifier
email	String	Unique user email
password	String	Hashed password using Werkzeug
role	String	User role (user or admin)

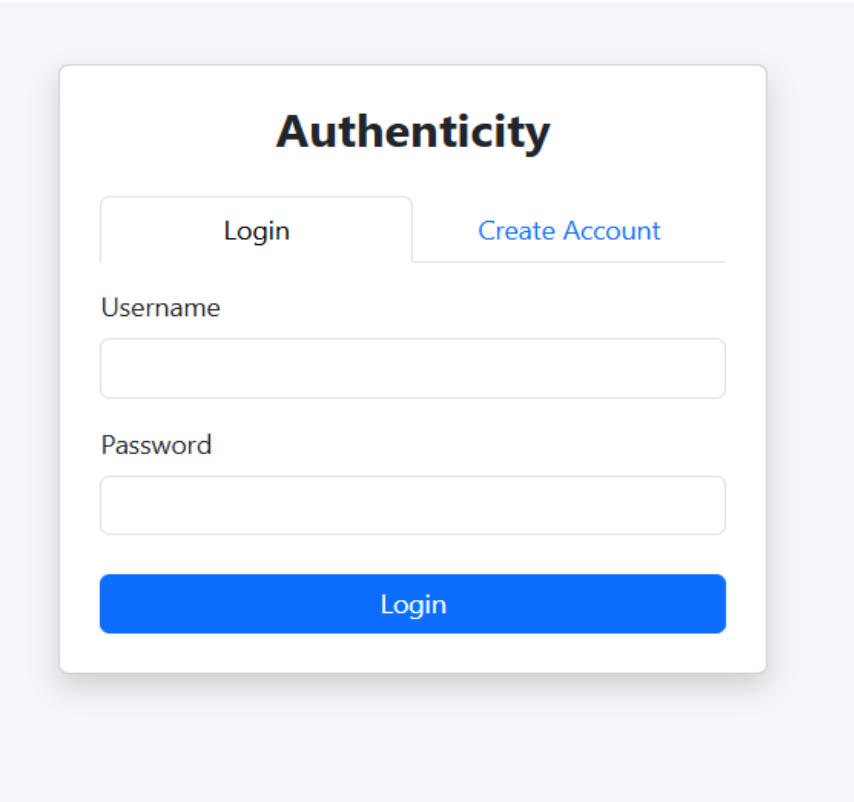
Document table

Column	Type	Description
id	Integer	Primary key (auto-increment)
user_id	Integer	Foreign key to User.id
filename	String	Uploaded file name
file_type	String	File format (txt/pdf/docx)
upload_time	DateTime	Timestamp of upload
original_text	Text	Extracted raw text from uploaded file
plagiarism_score	Float	Plagiarism similarity score (percentage)
ai_generated_prob	Float	AI-generated content probability (0–100)
similarity_report	Text/JSON	Structured similarity report, including matched segments for review

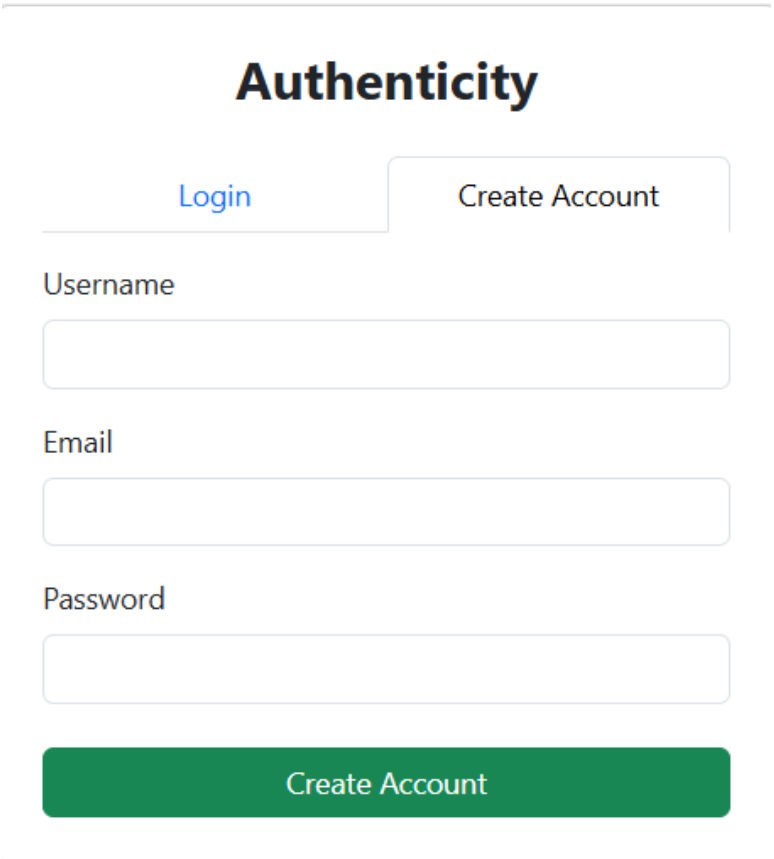
Appendix B - Sample Output

Sample outputs include screenshot references for user dashboard lists, document detail pages with scores, admin analytics charts, and similarity heatmap visualizations. These samples help evaluators understand expected behavior and validate the outputs.

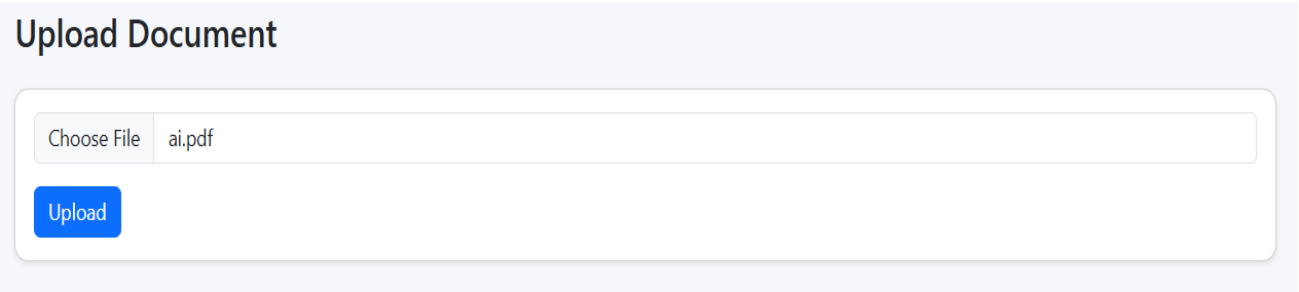
Login page:



Create Account:



Upload Document page:



Dashboard page:

Uploaded Documents

Upload Document

#	Filename	Plagiarism	AI Probability	Text Length	Action
1	ai.pdf	0.00%	48.88%	783	View
2	human.pdf	0.00%	0.00%	141	View
3	humanized_content.pdf	0.00%	51.90%	473	View
4	ai.pdf	100.00%	48.88%	783	View

Summary Page:

ai.pdf

Plagiarism Score: 100.0%

AI Probability: 48.88%

Matched Sentences

Sentence	Source Document	Similarity
artificial intelligence systems are computational frameworks designed to simulate aspects of human cognition through structured algorithms and datadriven models	ai.pdf	100.0%
these systems operate by analyzing large volumes of input data identifying patterns and generating outputs based on learned relationships rather than explicit programming for each task	ai.pdf	100.0%
modern ai architectures particularly those utilizing machine learning and deep learning techniques rely on iterative training processes to optimize performance metrics and improve predictive accuracy over time	ai.pdf	100.0%
as a result ai is capable of performing complex tasks such as natural language processing image recognition and decision making with a high degree of efficiency consistency and scalability across diverse application domains	ai.pdf	100.0%